EcoBuck Smart Compost Monitoring System

Connectivity & Backend (Assignment C) - Architectural Design Review

Author: Toqeer Ahmed
Role: Principal IoT Systems & Backend Architect
Date: July 8, 2026
Status: PROPOSED (Architecture Review Stage)

---

1. Project Overview

EcoBuck is a smart composting system developed by EcoZindagi designed to transform household and institutional kitchen food waste into high-quality organic compost. By monitoring and controlling internal compost conditions, EcoBuck replaces guesswork with empirical, data-driven insights, ensuring optimal decomposition, minimizing odor, reducing kitchen mess, and lowering landfill dependence.

The physical EcoBuck device utilizes an ESP32 microcontroller, temperature probes, and humidity/moisture sensors to track the composting cycle. The role of the **Connectivity & Backend** module is to establish a secure, reliable, and scalable bridge between these field-deployed physical devices and the user-facing mobile application.

---

2. Scope of Assignment

This architectural specification addresses **Assignment C: Connectivity & Backend**. It defines how device telemetry is ingested, stored, secured, queried, and visualized. The boundary of this module lies strictly between: * **The Firmware Client (Assignment B):** Consuming the structured JSON telemetry payloads generated by the ESP32 firmware. * **The Mobile Application (Assignment D):** Exposing REST/WebSocket APIs that drive the mobile-first dashboard, alerts interface, and historical data charts.

3. System Responsibilities

3.1 What Has Already Been Completed (Firmware Team B)

The firmware team has completed a local simulation architecture running Python 3 (validated via `pytest` and `unittest`), consisting of: * **Sensor Simulation (`sensors.py`):** Generates temperature and humidity readings with configurable noise, missing values, and calibration corrections. * **Data Filtering (`filters.py`):** A 5-sample rolling median filter to suppress high-frequency noise, a 10-sample rolling average trend calculator, and outlier detection (rejecting temperature jumps > 8°C). * **Finite State Machine (`state_machine.py`):** Determines compost status (`boot_self_test`, `active_composting`, `cooling_stabilizing`, `ready_candidate`, `ready_confirmed`, `needs_attention`) based on physical boundaries (e.g., active composting requires Temp > Ambient + 10°C; readiness requires stable ambient temp and 40-60% humidity for 2.5 days after a cycle age of 14 days). * **Payload Structuring (`payload.py`):** Packages metadata, raw/filtered readings, quality flags, battery levels, and state into a versioned JSON packet.

3.2 Where My Backend Responsibilities Begin

Backend responsibilities begin the moment the JSON packet leaves the EcoBuck device's network card. The backend must handle ingestion, authentication, database persistence, state transitions verification, and mobile app delivery.

3.3 What Responsibilities Belong to Assignment C

- **Protocol Selection & Ingestion Engine:** Designing a scalable, secure channel (HTTPS/MQTT) to accept telemetry.
- **Device Registry:** Keeping track of active devices, associated hardware IDs, current ownership (mapping devices to users), and active composting cycles.
- **Storage Architecture (Database):** Designing a schema separating high-frequency time-series telemetry from the transactional latest-state cache.
- **Rules & Alert Engine:** Processing incoming data stream to detect anomalies (e.g., device offline, sensor malfunction, critical heat warnings) and dispatching real-time notifications.
- **Mobile App API Gateway:** Serving authenticated endpoints for real-time dashboards, historical trends, alert logs, and manual state transitions (e.g., confirming compost readiness).
- **Security & Identity Governance:** Designing role-based access rules preventing unauthorized cross-device reads or writes.
- **Hardware Simulator:** Packaging a mock device script that can target the endpoint to enable parallel mobile development.

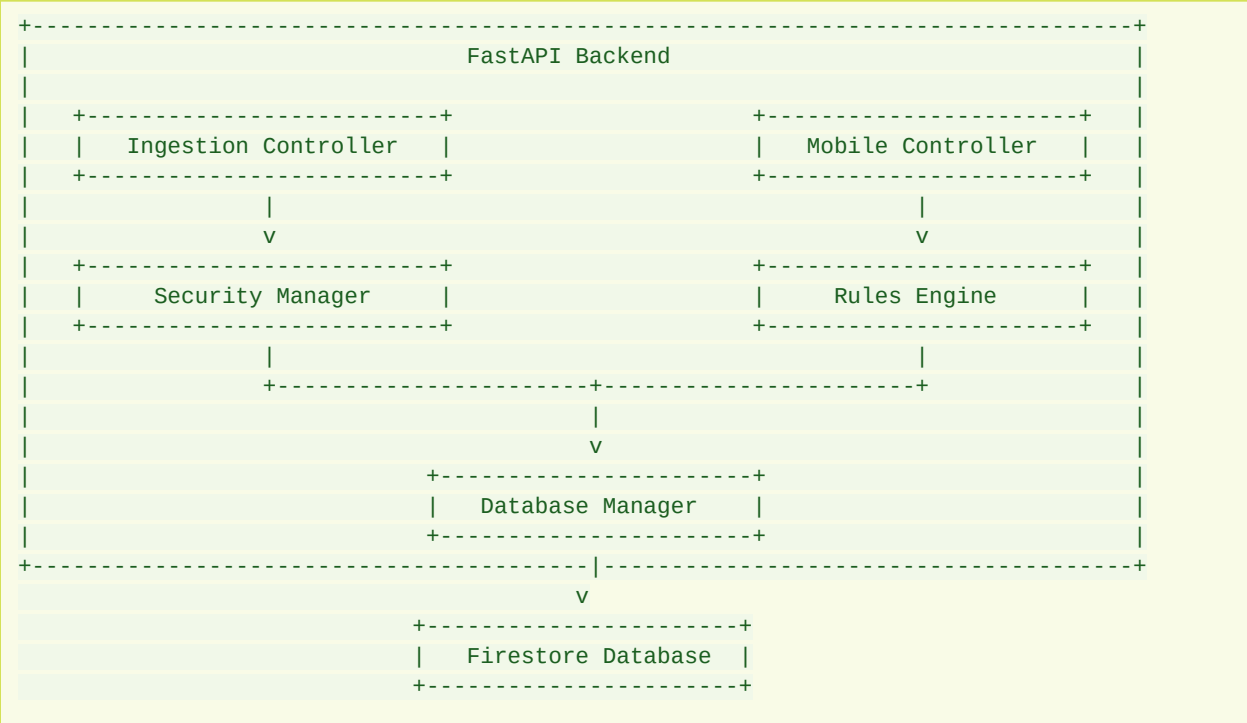
4. High-Level Architecture

The EcoBuck Backend uses a **Hybrid Microservices-lite architecture** utilizing **Python (FastAPI)** for the web layer, and **Google Cloud Firestore** (or **PostgreSQL + TimescaleDB**) for database storage.

FastAPI is selected due to its asynchronous nature (handling high-concurrency IoT requests), auto-generated OpenAPI documentation, and alignment with the firmware team's Python codebase.

5. Component Diagram

The backend application modules are decoupled to isolate failures and maintain modular testing boundaries:



6. Data Flow Diagram

1. **ESP32 Device / Simulator** sends an authenticated `POST /api/v1/telemetry` payload.
2. **Security Manager** authenticates the device API token.
3. **Database Manager** writes the time-series telemetry record to the database.

4. **Rules & Alert Engine** evaluates thresholds (e.g., temperature out of bounds, sensor failure, or transition to `ready_candidate`).
5. **Firebase Cloud Messaging** dispatches push notifications to the user's mobile app if an anomaly is detected.
6. **Mobile App Client** fetches the current status from the backend API gateway to update the dashboard.

---

7. Communication Flow

To establish a clear communication model, we select **HTTPS/REST** for device-to-cloud ingestion during the MVP phase, with a clean path to support **MQTT** as scaling demands grow.

7.1 Device to Backend

- **Protocol:** HTTPS
- **Endpoint:** `POST /api/v1/telemetry`
- **Authentication:** Custom HTTP Header `X-Device-Token` (cryptographically derived, per-device API key).
- **Configuration Sync:** `GET /api/v1/devices/config` to allow firmware to pull thresholds, calibration offsets, and sleep timers dynamically.

7.2 Backend to Mobile App

- **Protocol:** HTTPS (REST) & WebSockets (Real-time updates)
- **Authentication:** Authorization Header `Bearer <JWT_TOKEN>`.
- **Notification Protocol:** Firebase Cloud Messaging (FCM) for low-power, system-level background push alerts.

---

8. Backend Modules

8.1 Ingestion Service

Responsible for parsing, validating, and throttling input from field devices. * **Validation Rules:** * Verify `schema_version` matches current parser configurations. * Validate format structure using Pydantic

models. * Confirm timestamps are within reasonable tolerances (± 5 minutes of server time) to prevent garbage dates from broken RTC (Real-Time Clock) modules on the ESP32. * Reject payloads with `quality_flag` indicating `out_of_range` before saving, routing to error logs instead.

8.2 Database Schema & Model Design

For the production-ready NoSQL database design, see the detailed Firestore database specification: * firestore_design.md - Outlining users, devices, telemetry subcollections, latest status caching patterns, active alert indices, and notification history audit logs.

8.3 Alert & Rules Engine

Runs inline with ingestion to process anomalies. The core alerts map to firmware states and physical conditions: 1. **Offline Alarm (Heartbeat monitoring):** If a device's `last_seen` timestamp is older than `2 * config.READ_INTERVAL_SECONDS` (or `SUMMARY_INTERVAL_SECONDS`), trigger an offline alert. 2. **Sensor Error Alert:** If `quality_flag` is set to `sensor_error` or `missing`, flag a physical hardware check required alert. 3. **Temperature Hazard Alert:** If `filtered_temp` is greater than 65.0°C (indicates risk of destroying beneficial microbes) or exceeds safety thresholds. 4. **Moisture Extremes Alert:** If `raw_humidity` drops below 20% (compost ceases biological activity due to dry conditions) or climbs above 75% (anaerobic conditions, resulting in severe foul odor). 5. **Compost Maturity Candidate Alert:** Triggered when the device transitions to `ready_candidate`. The backend captures this and flags it in the user's dashboard, initiating the workflow for the user to validate compost readiness physically.

---

9. Integration Points

9.1 Firmware Integration Interface (API Contracts)

Telemetry Ingestion Endpoint

- **Method:** `POST`
- **Path:** `/api/v1/telemetry`
- **Payload Schema:** Refer to `/api/v1/telemetry` specification in firestore_design.md.

9.2 Mobile Application Integration Interface

Dashboard State Endpoint

- **Method:** `GET`
- **Path:** `/api/v1/devices/{device_id}/latest`

Manual Validation Endpoint

- **Method:** `POST`
- **Path:** `/api/v1/devices/{device_id}/validate`

---

10. Risks & Mitigations

10.1 Telemetry Flooding & Cloud Costs

- **Risk:** ESP32 devices misbehave or loop rapidly, sending millions of telemetry logs, exhausting database storage, and escalating cloud hosting costs.
- **Mitigation:** Rate Limiting at the API Gateway level (max 2 writes per minute per device).

10.2 Broken Real-Time Clocks (RTC) & Bad Timestamps

- **Risk:** ESP32 microcontrollers lose system time after power resets, writing telemetry packets dated to Unix Epoch (1970-01-01), corrupting time-series trends and dashboards.
- **Mitigation:** Verify incoming timestamps. If they deviate by >10 minutes, override with current server time.

10.3 Insecure Device Spoofing

- **Risk:** Attackers sniff device payloads and inject fake telemetry data or forge alerts, triggering panic notices on user devices.
- **Mitigation:** Enforce TLS 1.3 and unique per-device API tokens.

---

11. Assumptions & Dependencies

- **Network Availability:** ESP32 has local 2.4 GHz Wi-Fi connectivity.
- **Compost Cycle Length:** Typically between 14 to 60 days.
- **Dependencies:** Google Cloud Firebase / Firestore, Firebase Cloud Messaging (FCM).

---

12. Future AI Integration

1. **Predictive Compost Maturity Forecasting:** Instead of waiting for a fixed age (14 days) and stable state (2.5 days), a regression model (e.g., LSTM or XGBoost) can analyze temperature and humidity curves during the active composting phase to forecast the exact day the compost will become ready. This allows the mobile app to show a progression percentage (e.g., *"Your compost is 68% complete, estimated ready in 4 days"*).
2. **Odor & Anaerobic Detection Model:** By mapping changes in temperature trends against humidity drops, the backend can predict if the pile is becoming compacted and turning anaerobic (which releases high levels of odor). The system can warn the user: *"Odor alert: moisture patterns suggest anaerobic decay. Please turn your compost pile to introduce oxygen."*
3. **Anomalous Microbe Inactivation Alerts:** If the active phase fails to reach the required path-destroying heat levels (55°C to 60°C) or drops too fast, an anomaly detector can alert the user that pathogen destruction might be incomplete, advising them to add nitrogen-rich waste (greens) to restart active heating.
4. **Dynamic Threshold Adaptation:** Using reinforcement learning or historical cluster analysis, the system can learn the ambient patterns of the specific household (e.g., indoor kitchen vs. outdoor shed vs. balcony composters) and dynamically adapt calibration offsets and ambient thresholds per device.